University Of Engineering and Technology, Lahore
Computer Engineering Department

Lab Evaluation:

Course Name: Object Oriented Programming	**Course Code:** CMPE 221L
Assignment Type: Lab	**Dated:**
Semester: 2nd	**Session:** 2025(s)
Lab/Project/Assignment #: 7	**CLOs to be covered:** CLO 2
Lab Title: Associations	**Teacher Name:** Hafiz Muhammad Mubashar

CLO 2	Apply composition, inheritance and polymorphism and language extension concepts to build classes					
Levels (Marks)	**Level 1**	**Level 2**	**Level 3**	**Level 4**	**Level 5**	**Level 6**
(10)						
					Total	**/10**

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	**9-10**	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	**7-8**	L2	Submitted the lab tasks but have good understanding
Good	**5-6**	L3	Submitted the lab tasks but have weak understanding.
Basic	**3-4**	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	**1-2**	L5	Submitted only one lab task.
Not Acceptable	**0**	L6	Did not attempt

LAB No. 7

Lab Goals/Objectives:

- Basic OO programming in java
- Objects and Classes
- Associations

```java
// Task_09
public class Main {
    public static void main(String[] args) {
        Bank hbl = new Bank("HBL");
        Bank ubl = new Bank("UBL");

        Client ali = hbl.addClient(new Person("Ali Khan", "35202-1111111-1", "0300-1111111"));
        Client sara = hbl.addClient(new Person("Sara Ahmed", "35202-2222222-2", "0300-
2222222"));

        System.out.println("--- Adding accounts ---");
        hbl.addAccount("H-101", 5000, ali);
        hbl.addAccount("H-102", 12000, ali);
        hbl.addAccount("H-201", 8000, sara);
        hbl.addAccount("H-101", 100, sara);

        ubl.addClient(ali);
        ubl.addAccount("U-301", 3000, ali);

        System.out.println("\n--- Client details ---");
        System.out.println(ali);
        System.out.println(sara);

        System.out.println("--- Deposit / Withdraw ---");
        System.out.println("Deposit 1000 to H-101: " + ali.deposit(1000, "H-101"));
        System.out.println("Withdraw 2500 from H-102: " + ali.withdraw(2500, "H-102"));
        System.out.println("Withdraw 99999 from H-101: " + ali.withdraw(99999, "H-101"));
        System.out.println("Deposit to unknown account: " + ali.deposit(50, "X-000"));

        System.out.println("\n--- Totals ---");
        System.out.println("Ali total (all banks): " + ali.totalAmount());
        System.out.println("HBL total: " + hbl.totalAmount());
        System.out.println("UBL total: " + ubl.totalAmount());

        System.out.println("\n--- Searching ---");
        System.out.println(hbl.searchAccount("H-201"));
        System.out.println("Search missing account: " + hbl.searchAccount("Z-999"));
        System.out.println(hbl.searchCustomerDetail("35202-2222222-2"));
        System.out.println("Search missing CNIC: " + hbl.searchCustomerDetail("00000-0000000-
0"));

        System.out.println("\n--- Removing Ali from HBL ---");
        System.out.println("Removed: " + hbl.removeClient(ali.getId()));
        System.out.println(hbl);
        System.out.println(ali);
        System.out.println("Removing again: " + hbl.removeClient(ali.getId()));

        System.out.println("\n--- Bank summaries ---");
        System.out.println(hbl);
        System.out.println(ubl);
    }
}
```

```java
// Task_01
import java.util.ArrayList;
import java.util.List;

class Bank {
    private String name;
    private List<Client> clients = new ArrayList<>();
    private List<Account> accounts = new ArrayList<>();

    public Bank(String name) {
        this.name = name;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public List<Client> getClList() { return clients; }
    public void setClList(List<Client> clList) {
        this.clients = (clList != null) ? clList : new ArrayList<>();
    }

    public List<Account> getAcList() { return accounts; }
    public void setAcList(List<Account> acList) {
        this.accounts = (acList != null) ? acList : new ArrayList<>();
    }

    // Registers a new client from a Person; if that CNIC is already a client, returns the existing
one
    public Client addClient(Person p) {
        if (p == null) {
            System.out.println("Person cannot be null.");
            return null;
        }
        Client existing = searchCustomerDetail(p.getCnic());
        if (existing != null) {
            System.out.println("Client with CNIC " + p.getCnic() + " already exists in " + name +
".");
            return existing;
        }
        Client c = new Client(p);
        clients.add(c);
        return c;
    }

    // Registers an existing client (e.g. one who also banks elsewhere)
    public Client addClient(Client c) {
        if (c != null && !clients.contains(c)) {
            clients.add(c);
        }
        return c;
    }

    public Account addAccount(String id, float amount, Client c) {
        if (c == null) {
            System.out.println("Client cannot be null.");
```

```
            return null;
        }
        if (searchAccount(id) != null) {
            System.out.println("Account " + id + " already exists in " + name + ".");
            return null;
        }
        addClient(c);
        Account a = new Account(id, amount, c);
        accounts.add(a);
        c.addAccount(a);
        return a;
    }

    public Account searchAccount(String id) {
        for (Account a : accounts) {
            if (id != null && id.equals(a.getNumber())) return a;
        }
        return null;
    }

    public boolean removeClient(String id) {
        Client target = null;
        for (Client c : clients) {
            if (c.getId().equals(id)) { target = c; break; }
        }
        if (target == null) return false;

        List<Account> toRemove = new ArrayList<>();
        for (Account a : accounts) {
            if (a.getACholder() == target) toRemove.add(a);
        }
        accounts.removeAll(toRemove);
        target.getAcList().removeAll(toRemove);
        clients.remove(target);
        return true;
    }

    public float totalAmount() {
        float total = 0;
        for (Account a : accounts) {
            total += a.getAmount();
        }
        return total;
    }

    public Client searchCustomerDetail(String cnic) {
        for (Client c : clients) {
            Person p = c.getPersonDetails();
            if (p != null && cnic != null && cnic.equals(p.getCnic())) return c;
        }
        return null;
    }

    @Override
    public String toString() {
        return "Bank: " + name + " | Clients: " + clients.size()
                + " | Accounts: " + accounts.size() + " | Total: " + totalAmount();
    }
```

```
}
```

```
// Task_02
class Account {
    private static int count = 0;

    private String number;
    private float amount;
    private Client holder;

    // Default account: auto-generated number, zero balance, no holder
    public Account() {
        this(0f, null);
    }

    // Account with an auto-generated number
    public Account(float amount, Client holder) {
        count++;
        this.number = "A" + count;
        this.amount = Math.max(amount, 0f);
        this.holder = holder;
    }

    public Account(String number, float amount, Client holder) {
        count++;
        this.number = number;
        this.amount = Math.max(amount, 0f);
        this.holder = holder;
    }

    public String getNumber() { return number; }
    public void setNumber(String number) { this.number = number; }

    public float getAmount() { return amount; }
    public void setAmount(float amount) { this.amount = amount; }

    public Client getACholder() { return holder; }
    public void setACholder(Client holder) { this.holder = holder; }

    public static int getCount() { return count; }

    // Returns the new balance, or -1 if the withdrawal failed
    public float withdraw(float amt) {
        if (amt <= 0) {
            System.out.println("Invalid withdrawal amount.");
            return -1;
        }
        if (amt > amount) {
            System.out.println("Insufficient balance in account " + number + ".");
            return -1;
        }
        amount -= amt;
        return amount;
```

```
    }

    // Returns the new balance, or -1 if the deposit failed
    public float deposit(float amt) {
        if (amt <= 0) {
            System.out.println("Invalid deposit amount.");
            return -1;
        }
        amount += amt;
        return amount;
    }

    @Override
    public String toString() {
        String holderName = (holder != null && holder.getPersonDetails() != null)
                ? holder.getPersonDetails().getName() : "N/A";
        return "Account No: " + number + ", Balance: " + amount + ", Holder: " + holderName;
    }
}
```

```
// Task_03
import java.util.ArrayList;
import java.util.List;

class Client {
    private static int count = 0;

    private String id;
    private Person personDetails;
    private List<Account> accounts = new ArrayList<>();

    public Client(Person personDetails) {
        count++;
        this.id = "C" + count;
        this.personDetails = personDetails;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public Person getPersonDetails() { return personDetails; }
    public void setPersonDetails(Person personDetails) { this.personDetails = personDetails; }

    public List<Account> getAcList() { return accounts; }
    public void setAcList(List<Account> acList) {
        this.accounts = (acList != null) ? acList : new ArrayList<>();
    }

    public static int getCount() { return count; }

    public float totalAmount() {
        float total = 0;
        for (Account a : accounts) {
```

```
            total += a.getAmount();
        }
        return total;
    }

    public void addAccount(Account a) {
        if (a != null && !accounts.contains(a)) {
            accounts.add(a);
        }
    }

    private Account findAccount(String accNo) {
        for (Account a : accounts) {
            if (accNo != null && accNo.equals(a.getNumber())) return a;
        }
        return null;
    }

    // Returns the new balance, or -1 if the account is not found / withdrawal fails
    public float withdraw(float amount, String accNo) {
        Account a = findAccount(accNo);
        if (a == null) {
            System.out.println("Account " + accNo + " not found for client " + id + ".");
            return -1;
        }
        return a.withdraw(amount);
    }

    // Returns the new balance, or -1 if the account is not found / deposit fails
    public float deposit(float amount, String accNo) {
        Account a = findAccount(accNo);
        if (a == null) {
            System.out.println("Account " + accNo + " not found for client " + id + ".");
            return -1;
        }
        return a.deposit(amount);
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("Client ID: ").append(id).append("\n");
        sb.append(personDetails).append("\n");
        if (accounts.isEmpty()) {
            sb.append("  No accounts.\n");
        } else {
            for (Account a : accounts) {
                sb.append("  Account ID: ").append(a.getNumber())
                  .append(", Amount: ").append(a.getAmount()).append("\n");
            }
        }
        return sb.toString();
    }
}
```

Output:

```
--- Adding accounts ---
Account H-101 already exists in HBL.
```

```
--- Client details ---
Client ID: C1
Name: Ali Khan, CNIC: 35202-1111111-1, Phone: 0300-1111111
  Account ID: H-101, Amount: 5000.0
  Account ID: H-102, Amount: 12000.0
  Account ID: U-301, Amount: 3000.0

Client ID: C2
Name: Sara Ahmed, CNIC: 35202-2222222-2, Phone: 0300-2222222
  Account ID: H-201, Amount: 8000.0
```

```
--- Deposit / Withdraw ---
Deposit 1000 to H-101: 6000.0
Withdraw 2500 from H-102: 9500.0
Insufficient balance in account H-101.
Withdraw 99999 from H-101: -1.0
Account X-000 not found for client C1.
Deposit to unknown account: -1.0
```

```
--- Totals ---
Ali total (all banks): 18500.0
HBL total: 23500.0
UBL total: 3000.0
```

```
--- Searching ---
Account No: H-201, Balance: 8000.0, Holder: Sara Ahmed
Search missing account: null
Client ID: C2
Name: Sara Ahmed, CNIC: 35202-2222222-2, Phone: 0300-2222222
  Account ID: H-201, Amount: 8000.0

Search missing CNIC: null
```

```
--- Removing Ali from HBL ---
Removed: true
Bank: HBL | Clients: 1 | Accounts: 1 | Total: 8000.0
Client ID: C1
Name: Ali Khan, CNIC: 35202-1111111-1, Phone: 0300-1111111
  Account ID: U-301, Amount: 3000.0

Removing again: false
```

```
--- Bank summaries ---
Bank: HBL | Clients: 1 | Accounts: 1 | Total: 8000.0
Bank: UBL | Clients: 1 | Accounts: 1 | Total: 3000.0

Process finished with exit code 0
```